

Estudio de Cobertura Wifi en la Banda 2.4 GHz en la Facultad de Ingeniería Electrónica y Eléctrica (Pabellón B) usando el método de Interpolación Espacial Cúbica y Python

Zamora Asenjo, Roly Willans, roly.zamora@unmsm.edu.pe

Introducción y Justificación Experimental:

La cobertura de redes inalámbricas en entornos verticales, como la Facultad de Ingeniería Electrónica y Eléctrica, representa un desafío técnico debido a la atenuación de la señal provocada por barreras estructurales como muros y los pisos. En este trabajo se aborda el análisis de cobertura Wi-Fi en la banda de 2.4 GHz para las señales “Bitel Electrónica” y “Alumnos UNMSM” en la FIEE, pabellón B. Para ello, se implementa un método de interpolación espacial cúbica sobre datos de intensidad de señal recibida (RSSI) recolectados en puntos estratégicos por piso. El objetivo es generar mapas de calor que permitan identificar visualmente las "zonas muertas" ($\text{RSSI} < -75 \text{ dBm}$) en cada piso del pabellón, determinando así las ubicaciones óptimas para la instalación de repetidores o puntos de acceso adicionales que garanticen la disponibilidad del servicio en todo el edificio.

Esquema y Fotografía

La recolección de datos se llevó a cabo utilizando una aplicación móvil especializada en análisis de redes Wi-Fi, instalada en un dispositivo smartphone redmi 13C. Esta herramienta permite medir en tiempo real la intensidad de señal recibida (RSSI), expresada en decibelios relativos a 1 milivatio (dBm), de los puntos de acceso detectados en el entorno. El procedimiento consistió en recorrer cada uno de los tres pisos de la facultad, deteniéndose en 13 puntos estratégicos predefinidos que se repiten por piso, distribuidos a la misma distancia entre si para cubrir pasillos y zonas comunes. En cada punto, se registró la coordenada espacial (x, y) en metros, el número de piso y el valor del RSSI correspondiente para cada señal Wifi. Los datos obtenidos fueron exportados en formato CSV por separado para su posterior procesamiento y análisis mediante técnicas numéricas de interpolación espacial.



Figura 1. Vista satelital de la Facultad de Ingeniería Electrónica y Eléctrica Pabellón B (Fuente: Google Earth)

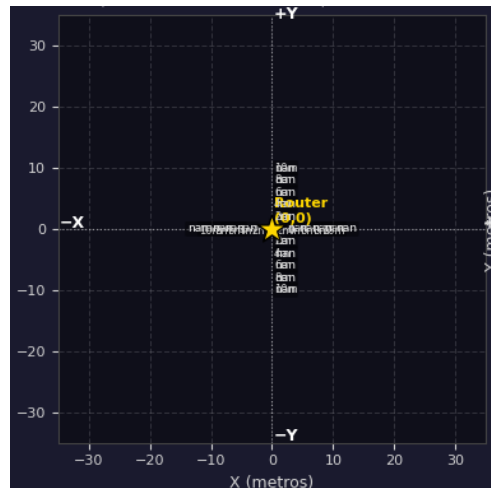


Figura 2. Coordenadas para la representación de cobertura RSSI mediante el programa de Python



Figura 3. Algunas fotografías de prueba sobre las mediciones reales de la red Wifi herramienta Wifi Analyzer

Datos Medidos e Incertidumbre

A continuación se muestra la tabla de datos recolectados mediante la aplicación Analyzer Wifi, brindando datos de rssi en dbm en distintos puntos en los tres pisos de la facultad de ingeniería electrónica y eléctrica, pabellón B.

En las lecturas se observa las redes Wifi “Alumnos FIEE (red protegida por login, cuenta institucional + contraseña)” y la red Wifi “Bitel 5G F. Electronica y Eléctrica” las cuales funcionan en la banda 2.4 GHz y 5 GHz, sin embargo para las lecturas se considero la de mayor alcance, en ese sentido solo la banda de 2.4 GHz.

Tabla 01: Datos experimentales de Posición vs. Dbm

X (metros)	Y (metros)	piso	Rssi (Bitel)	rssi (AlumnosFIEE)
7	7	3	-66	-86
7	0	3	-67	-96
0	-7	3	-61	-90

0	0	3	-85	-81
-7	0	3	-69	-76
-14	0	3	-77	-75
7	-7	3	-90	-68
7	-14	3	-78	-84
7	-21	3	-85	-76
7	-28	3	-90	-90
7	-35	3	-90	-92
0	-35	3	-90	-90
0	-28	3	-90	-89
0	-28	2	-90	-89
0	-35	2	-90	-87
7	-35	2	-71	-69
7	-28	2	-76	-78
7	-21	2	-78	-90
7	-14	2	-61	-62
7	-7	2	-80	-64
7	7	2	-80	-96
0	0	2	-90	-87
7	0	2	-90	-72
-7	0	2	-64	-67
-14	0	2	-77	-78
-7	-7	2	-68	-64
-7	-14	2	-90	-74
0	-7	2	-61	-80
7	7	1	-73	-76
7	0	1	-68	-90
-7	-7	1	-54	-62
-7	-14	1	-37	-74
-7	0	1	-73	-85
-14	0	1	-72	-90
0	0	1	-69	-90
-21	-7	1	-89	-63
-21	-14	1	-86	-68
0	-7	1	-66	-75
7	-7	1	-72	-72
7	-14	1	-78	-66
7	-21	1	-40	-90
7	-28	1	-68	-90
7	-35	1	-72	-86
0	-28	1	-88	-89
0	-35	1	-72	-83

Procesamiento de Datos mediante Métodos Numéricos

Interpolación Espacial Cúbica de Datos Dispersos

Objetivo: Obtener un mapa de calor de RSSI en dbm utilizando métodos numéricos y Python

Método aplicado:

La interpolación numérica es “la técnica utilizada para construir nuevos puntos de datos dentro del rango de un conjunto discreto de valores conocidos, proporcionando una forma de estimar el valor en ubicaciones no medidas dentro de los límites de los parámetros originales” (Wang, 2013). A diferencia de la interpolación polinómica de un solo grado alto para todo el conjunto de datos, la interpolación por splines ajusta polinomios de bajo grado a subconjuntos pequeños de los valores. El objetivo particular del spline cúbico es “lograr una función interpolante continua tanto en su primera como en su segunda derivada, tanto dentro de los intervalos como en los nodos, lo cual produce una curva visualmente más suave y sin cambios abruptos de pendiente” (Wang, 2013).

Sin embargo, el caso de este estudio no corresponde al spline cúbico clásico unidimensional, sino a una variante diseñada específicamente para datos dispersos en dos dimensiones. “La función “griddata” de la librería “SciPy”, con el parámetro “method='cubic'”, delega el cálculo a la clase “CloughTocher2Dinterpolator” para realizar interpolación cúbica por partes en 2D” (SciPy Developers, 2024).

Según la documentación oficial de esta librería, “el interpolante se construye triangulando los datos de entrada mediante la biblioteca Qhull, y luego se construye un polinomio de Bézier interpolante cúbico por partes en cada triángulo utilizando un esquema de Clough-Tocher, garantizando que el interpolante resultante sea continuamente diferenciable” (SciPy Developers, 2024). Adicionalmente, esta misma fuente señala que dado que “griddata” está basado en triangulación, resulta apropiado para datos dispersos y no estructurados, como el caso de las mediciones RSSI por piso en ubicaciones no uniformes dentro del pabellón B. El método no impone ninguna tendencia global sobre el conjunto de datos, construye la superficie interpolada solo a partir de la vecindad inmediata de cada punto

La elección metodológica es coherente con el objetivo del estudio, las mediciones de RSSI no se distribuyen en una malla regular, sino en puntos estratégicos irregulares definidos por la geometría de aulas, pasillos y zonas comunes de cada piso. El resultado del proceso de triangulación y ajuste cúbico permite generar una superficie continua de valores estimados de RSSI en todo el plano (x, y), a partir de la cual se construyen los mapas de calor y se identifican las zonas muertas (RSSI < -75 dBm) reportadas en la sección de resultados.

Formulas del código:

Interpolación cúbica de Clough-Tocher

Superficie de RSSI por triángulo (coordenadas baricéntricas)

$$P(u, v, w) = \sum_{i+j+k=3} b_{ijk} \cdot \frac{3!}{i! j! k!} \cdot u^i v^j w^k$$

Con $i+j+k=3$, $u+v+w=1$

Son (u,v,w) las coordenadas baricéntricas del punto dentro del triángulo formado por tres mediciones vecinas, b_{ijk} son coeficientes de control de Bézier (calculados a partir de los valores de RSSI y de sus gradientes estimados en cada vértice), y $P(u,v,w)$ es el valor interpolado de RSSI en ese punto del triángulo. El método triangula los puntos (x, y) medidos usando el algoritmo Qhull, luego resuelve este polinomio en cada triángulo por separado, así la superficie resultante es continuamente diferenciable (C^1) entre triángulos adyacentes.

En el código:

`zi_grid = griddata(puntos, rssi, (xi_grid, yi_grid), method='cubic', fill_value=rssi.min())`

Esta línea ejecuta internamente todo el proceso: triangula los pares (x, y) de la tabla de mediciones, ajusta el polinomio cúbico de Bézier en cada triángulo, y evalúa la superficie resultante en cada uno de los 200×200 puntos de la malla fina (xigrid, yigrid), produciendo el mapa de calor continuo de RSSI que luego se colorea con `contourf`.

RSSI medio por piso

Media aritmética de las mediciones

$$\overline{RSSI} = \frac{1}{n} \sum_{i=1}^n RSSI_i$$

Para $i = 1 \dots n$

n es el número de puntos medidos en piso y $RSSI_i$ es el valor de señal registrado en cada punto; un simple promedio aritmético que resume el nivel general de cobertura del piso.

En el código

`rssi_medio = rssi.mean()`

Se calcula sobre los valores medidos (no sobre la malla interpolada) y se muestra en el título de cada subplot, sirve como indicador de la calidad de cobertura de cada piso/red.

Máscara de zona muerta

Umbral binario sobre la malla interpolada

$$M(x, y) = \begin{cases} 1 & \text{si } Z(x, y) < \text{Umbral} \\ 0 & \text{en otro caso} \end{cases}$$

$Z(x,y)$ es el valor de RSSI interpolado en el punto (x,y) de la malla, Umbral = -75 dBm es el límite definido como zona muerta, y $M(x,y)$ es una máscara binaria que marca cada celda de la malla como zona muerta (1) o zona con cobertura aceptable (0).

En el código

`mascara_muerta = zi_grid < UMBRAL_ZONA_MUERTA`

Esta comparación vectorizada de NumPy se aplica a toda la malla interpolada y da un arreglo booleano del mismo tamaño que `zigrid`. Esta máscara se usa después para: a) dibujar la línea roja punteada de contorno en el umbral exacto, b) calcular el porcentaje de área en zona muerta, y c) ubicar el centro de masa de la región para dibujar el círculo indicador.

Porcentaje de área en zona muerta

Proporción de celdas bajo el umbral

$$\% \text{ Zona muerta} = \frac{\sum M(x, y)}{N_{total}} \times 100$$

$\sum M(x,y)$ es la cantidad de celdas de la malla marcadas como zona muerta, y N_{total} es el número total de celdas de la malla ($200 \times 200 = 40\,000$ en el script).

En el código

$$pct_muerta = np.sum(mascara_muerta) / mascara_muerta.size * 100$$

Se muestra en el título de cada subplot y es el indicador cuantitativo principal que resume, para cada piso y red, qué proporción del área total analizada queda por debajo del umbral de cobertura aceptable.

Radio equivalente del círculo de zona muerta

Despejado del área de un círculo, $A = \pi r^2$

$$r = \sqrt{\frac{A}{\pi}}$$

A es el área real (metros cuadrados) ocupada por la región de zona muerta detectada, obtenida al multiplicar el número de celdas marcadas por el área que representa cada celda de la malla. Esta fórmula no dibuja el contorno real e irregular de la zona muerta, sino un círculo de área equivalente, centrado en el centroide de la región, como referencia visual.

En el código

$$radio = np.sqrt(area_px / np.pi) * (xi[-1] - xi[0]) / mascara_muerta.shape[1]$$

Siendo área_px es el conteo de celdas en zona muerta (en unidades de píxeles de la malla); el factor (xi[-1]-xi[0])/shape[1] convierte ese conteo de píxeles a metros reales, según la escala de la malla (rango de 70 m dividido en 200 celdas). El resultado se usa para dibujar el círculo rojo translúcido (patches.Circle) sobre el mapa.

Centro de masa (centroide) de zona muerta

Centro de masa de la región marcada

$$x_c = \frac{\sum x_i M_i}{\sum M_i} \quad y_c = \frac{\sum y_i M_i}{\sum M_i}$$

La suma recorre las celdas (x_i, y_i) de la malla, M_i es el valor de la máscara de zona muerta (1 o 0) en esa celda. Es una máscara binaria, equivale al centroide geométrico simple de la región marcada: el promedio de las coordenadas de todas las celdas en zona muerta.

En el código

$$cy_px, cx_px = center_of_mass(mascara_muerta)$$

La función center_of_mass de scipy.ndimage calcula automáticamente estas coordenadas sobre la máscara booleana. El resultado (en índices de la malla) se convierte después a coordenadas reales en metros (real_cx, real_cy) y se usa como el centro donde se dibuja el círculo y la etiqueta de texto "Zona muerta".

Normalización de color

Mapeo de RSSI a la paleta de colores

$$c(x, y) = \frac{Z(x, y) - V_{min}}{V_{max} - V_{min}}, \quad c \in [0, 1]$$

$Z(x,y)$ es el valor de RSSI interpolado, y V_{min} , V_{max} son los extremos configurados de la escala de colores. El valor normalizado $c(x,y) \in [0,1]$ se usa para seleccionar el color dentro de la paleta continua 'RdYlGn_r' (rojo → amarillo → verde invertido).

En el código:

$$contour = ax.contourf(xi_grid, yi_grid, zi_grid, levels=50, cmap='RdYlGn_r', vmin=VMIN_ESCALA, vmax=VMAX_ESCALA, alpha=0.92)$$

Para v_{min} y v_{max} ($V_{MIN_FIJO} = -95$ y $V_{MAX_FIJO} = -55$) fijan los extremos de la escala para que la paleta de color se distribuya proporcional y entre el peor y mejor valor de señal esperado, evitando que un dato atípico comprima el resto de la escala en un solo color.

Resultado obtenido:

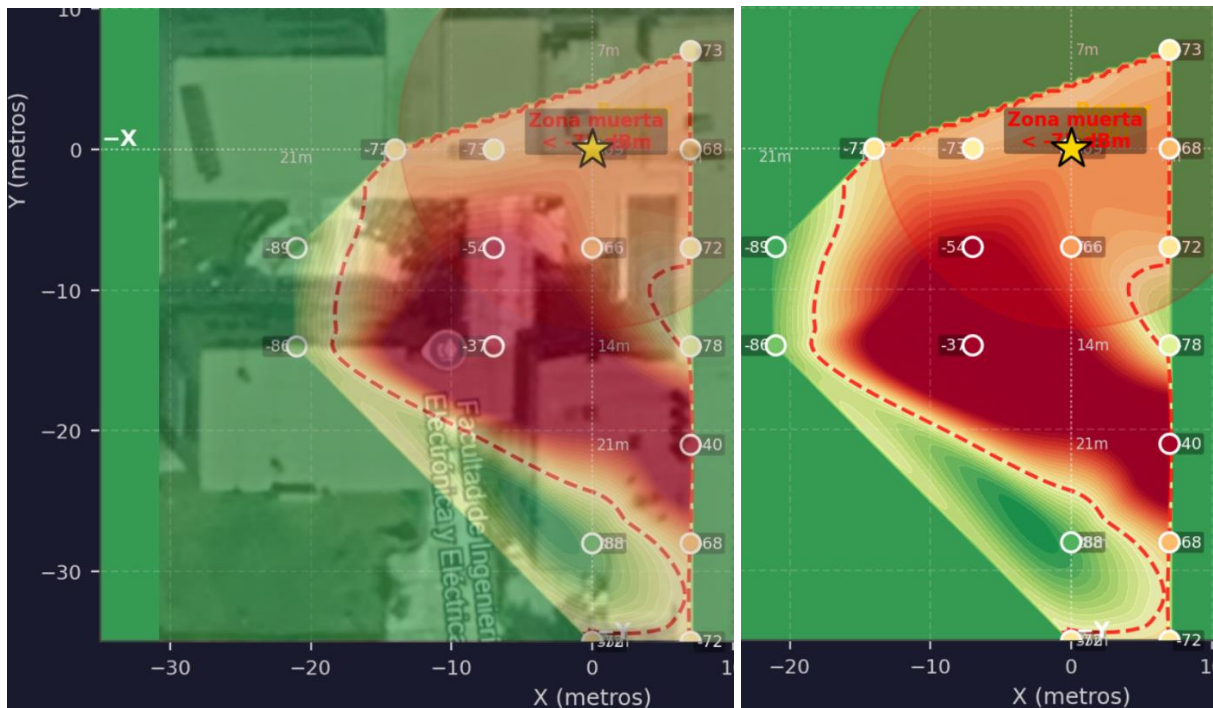


Figura 4. Contraste del Mapa de Cobertura de red Wifi 2.4 GHz, cuadrantes 1er piso correspondiente a Bitel

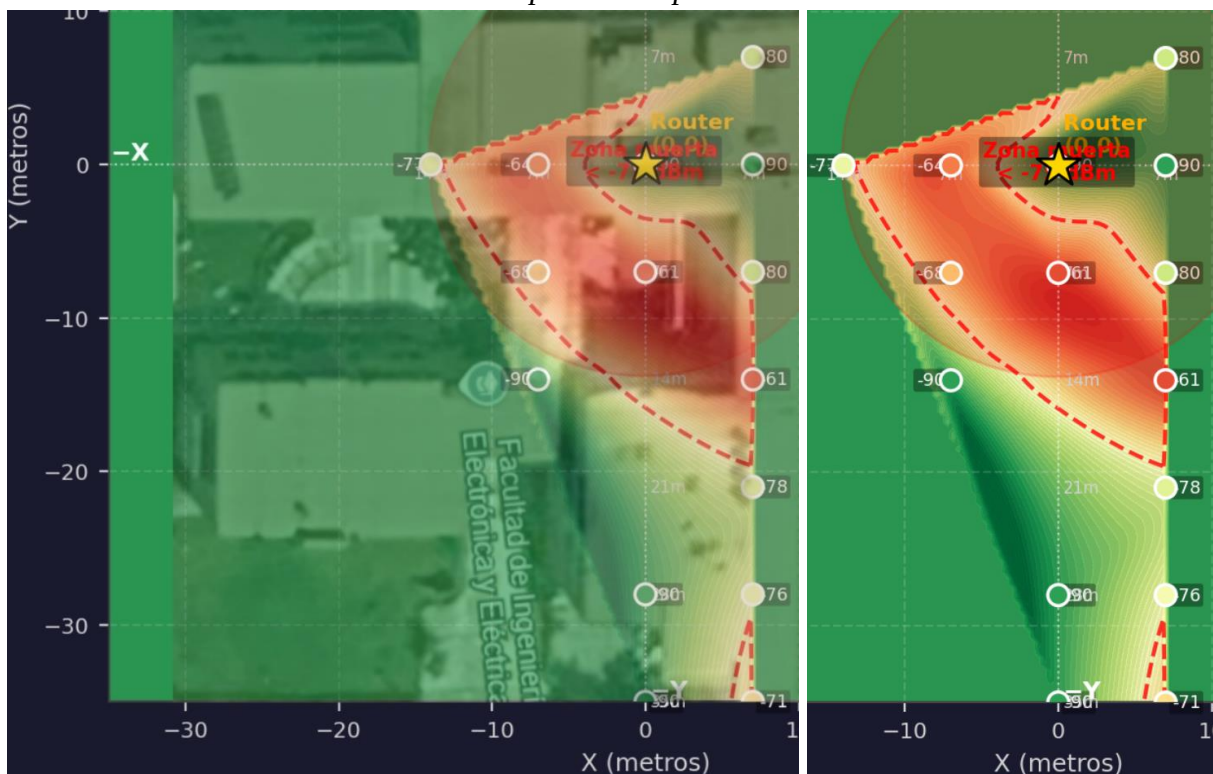


Figura 5. Contraste del Mapa de Cobertura de red Wifi 2.4 GHz, cuadrantes 2do piso correspondiente a Bitel

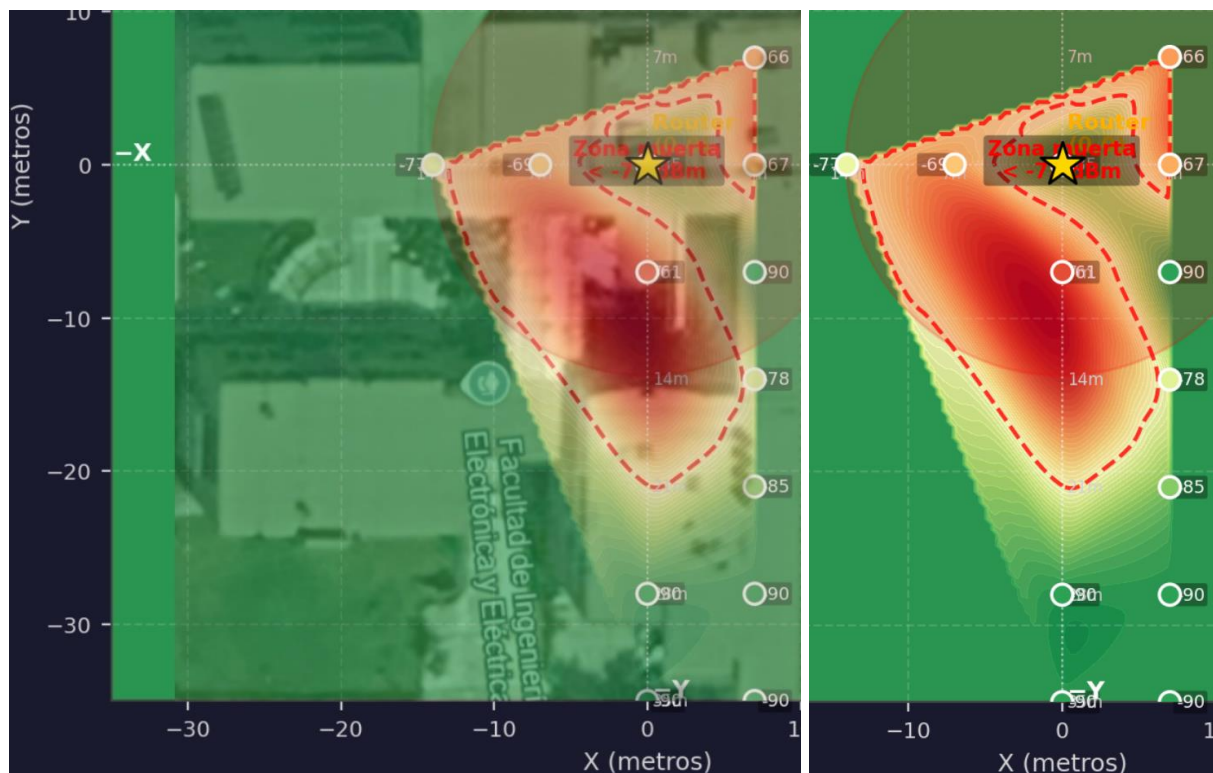


Figura 6. Contraste del Mapa de Cobertura de red Wifi 2.4 GHz, cuadrantes 3er piso correspondiente a Bitel

De acuerdo con los datos correspondientes en el piso 1 la RSSI medio para la red wifi de Bitel es -69.2 dbm, el piso 2 tiene un RSSI medio de -77.7 dbm y el piso 3 tiene un valor medio de -79.8 dBm.

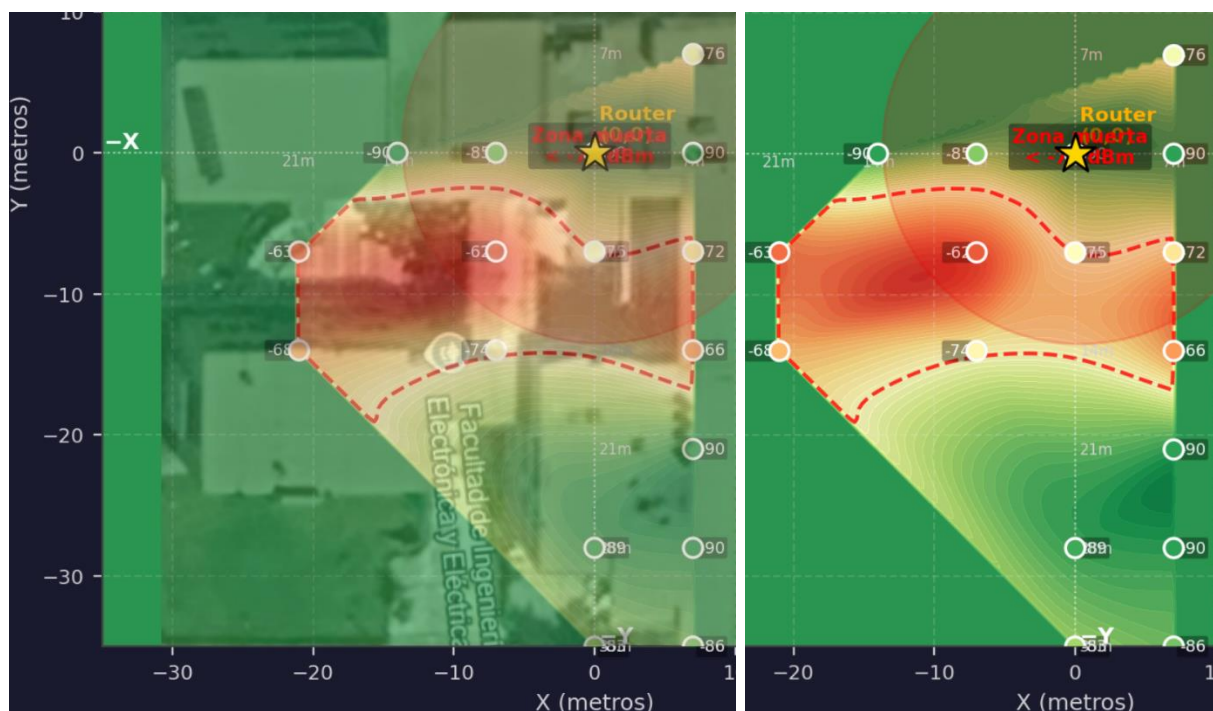


Figura 7. Contraste del Mapa de Cobertura de red Wifi 2.4 GHz, cuadrantes 1er piso correspondiente a Alumnos UNMSM

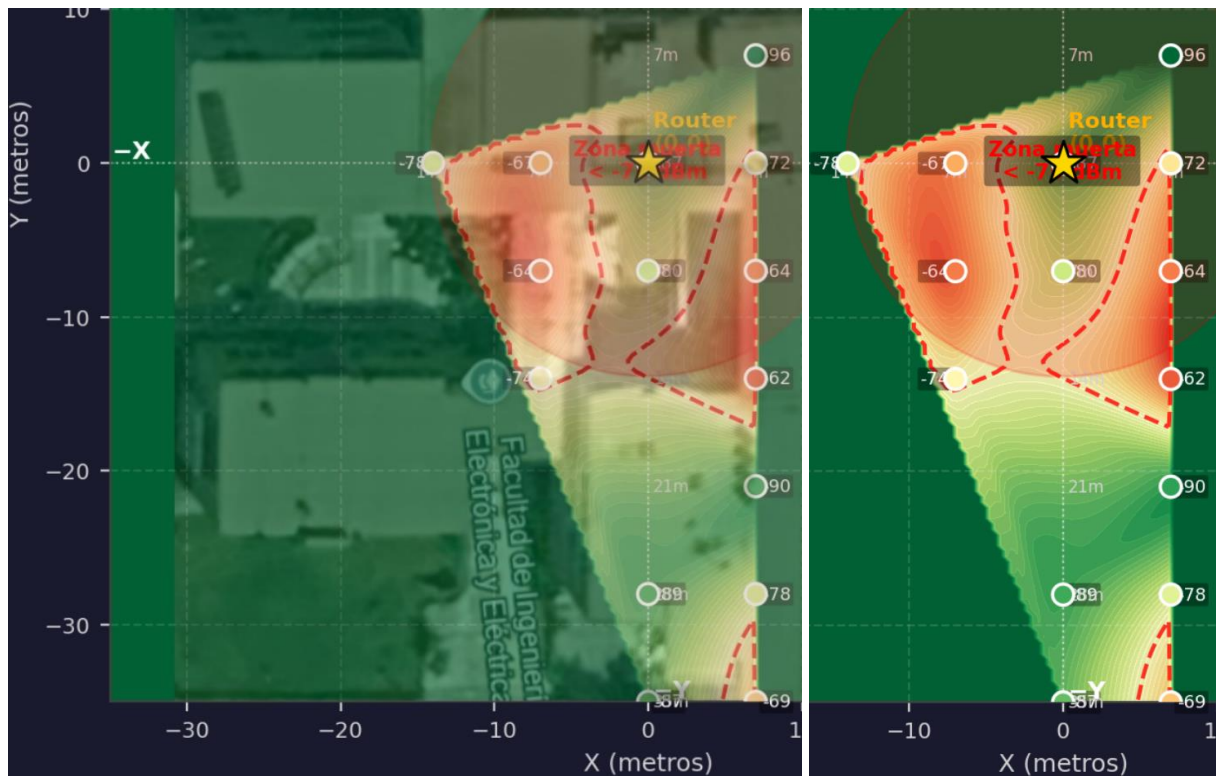


Figura 8. Contraste del Mapa de Cobertura de red Wifi 2.4 GHz, cuadrantes 2do piso correspondiente a Alumnos UNMSM

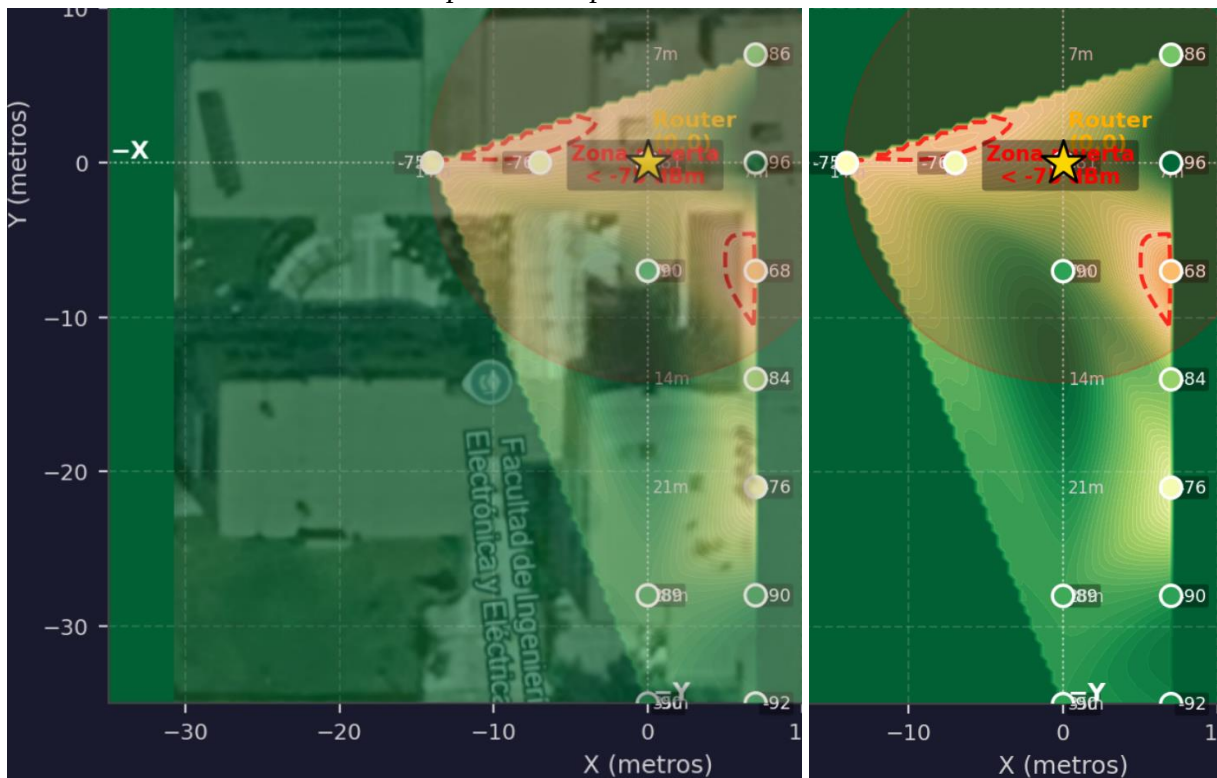


Figura 9. Contraste del Mapa de Cobertura de red Wifi 2.4 GHz, cuadrantes 3er piso correspondiente a Alumnos UNMSM

De acuerdo con los datos correspondientes en el piso 1 la RSSI medio para la red wifi Alumnos UNMSM es -79.4 dbm, el piso 2 tiene un RSSI medio de -77.1 dbm y el piso 3 tiene un valor medio de -84.1 dBm.

Análisis del Error:

El error de la investigación rodea los 5dBm debido a la sensibilidad del equipo utilizado.

Discusión:

Los resultados obtenidos muestran que, si bien la cobertura de ambas redes se complementa parcialmente en algunas zonas, ninguna logra garantizar por sí sola una señal aceptable en la totalidad del pabellón B. La red Bitel presenta un RSSI medio de -69.2 dBm en el piso 1, que se deteriora progresivamente a -77.7 dBm en el piso 2 y -79.8 dBm en el piso 3; un patrón similar, aunque menos uniforme, se observa en la red Alumnos UNMSM, con promedios de -79.4, -77.1 y -84.1 dBm respectivamente. Este comportamiento es consistente con la atenuación esperada por muros entre niveles, y confirma que la ubicación del punto de acceso principal no es suficiente para sostener cobertura en un edificio de tres pisos. Las zonas identificadas como más afectadas son la sala de lectura, el comedor y el centro deportivo que corresponden a áreas alejadas del router o separadas por múltiples obstáculos estructurales, lo cual coincide con el comportamiento esperado de atenuación en entornos interiores. En cuanto a la instrumentación utilizada, el smartphone (Redmi 13C) ofreció practicidad y portabilidad para el levantamiento rápido de datos en los tres pisos, resultando adecuado para los fines exploratorios de este estudio. Sin embargo, un equipo profesional como el analizador de espectro Narda SRM-3006 constituye una alternativa más rigurosa: al ser selectivo en frecuencia (9 kHz–6 GHz) y estar calibrado bajo normas internacionales, permite discriminar con precisión la contribución de cada fuente de radiofrecuencia mediante antenas isotrópicas y trazabilidad metrológica. La elección del smartphone en este trabajo respondió a un criterio de viabilidad práctica, aunque se reconoce que introduce una fuente adicional de incertidumbre en las mediciones.

Discusión de resultados:

El margen de error estimado en este estudio, de aproximadamente -5 dBm, es atribuible a la sensibilidad del smartphone utilizado frente a un equipo de medición profesional. Esta incertidumbre afecta el valor de cada lectura individual, pero no invalida las tendencias espaciales observadas en los mapas de calor, que es la información relevante para la toma de decisiones sobre la ubicación de repetidores o puntos de acceso. El método de interpolación cúbica aplicado es adecuado para el reducido número de puntos disponibles por piso y para los cambios abruptos de señal característicos de un entorno con múltiples obstáculos que permitió capturar los patrones de cobertura sin suavizar las discontinuidades reales provocadas por la infraestructura del edificio. Se recomienda que estudios futuros incrementen la densidad de puntos de muestreo por piso y usen equipos de medición para reducir la incertidumbre y validar con mayor rigor las decisiones de despliegue de infraestructura de red.

Conclusiones

A partir del análisis de interpolación espacial cúbica realizado sobre las mediciones de RSSI recolectadas en el pabellón B de la FIEE, se concluye que la infraestructura del edificio genera zonas de cobertura Wi-Fi insuficiente en los tres niveles, particularmente en la sala de lectura, el comedor y el centro deportivo, donde los valores de señal caen consistentemente por debajo del umbral crítico de -75 dBm en ambas redes evaluadas. El piso 3, pese a su cercanía relativa al punto de acceso principal, no logra un RSSI medio óptimo (Bitel: -79.8 dBm; Alumnos UNMSM: -84.1 dBm), lo que evidencia que la potencia de un único punto de acceso no basta para compensar la atenuación estructural del edificio. Se recomienda instalar al menos un repetidor o punto de acceso adicional por piso, priorizando las zonas identificadas como críticas en los mapas de calor generados. Asimismo, se concluye que la interpolación cúbica de datos dispersos (Clough-Tocher), implementada mediante la función `griddata` de la librería SciPy, constituye una herramienta numérica adecuada para este tipo de análisis exploratorio con densidad de muestreo limitada, dado que respeta las discontinuidades locales de la señal sin imponer una tendencia espacial global que podría enmascarar zonas muertas reales. Finalmente, se recomienda para trabajos futuros aumentar el número de puntos de medición por piso y emplear equipos de medición certificados, con el fin de reducir el margen de error estimado (~5 dBm) y fortalecer la validez estadística de las conclusiones obtenidas.

Bibliografía

- Wang, K. (2013). A study of cubic spline interpolation. InSight: Rivier Academic Journal, 9(2). https://www2.rivier.edu/journal/ROAJ-Fall-2013/J784-Wang_cubic-splines.pdf
- Wikipedia. (2025, 5 de diciembre). Spline interpolation. En Wikipedia. https://en.wikipedia.org/wiki/Spline_interpolation
- SciPy Developers. (2024). `scipy.interpolate.CloughTocher2DInterpolator` — SciPy v1.18.0 Manual. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.CloughTocher2DInterpolator.html>
- SciPy Developers. (2024). Scattered data interpolation (`griddata`) — SciPy v1.18.0 Manual. https://docs.scipy.org/doc/scipy/tutorial/interpolate/ND_unstructured.html

Apéndice A: Código Fuente (Scripts / Programas)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import griddata
from scipy.ndimage import label, center_of_mass
import matplotlib.patches as patches
ARCHIVOS = {
    'Bitel': 'datos_rssi_bitel.csv',
    'Alumnos FIEE': 'datos_rssi_alumnosunmsm.csv',
}
UMBRAL_ZONA_MUERTA = -75 # dBm – ajusta según tu criterio
RANGO_AUTOMATICO = False
VMIN_FIJO = -95
VMAX_FIJO = -55 # antes era -35; ajustado a un valor de señal "buena" realista
def leer_csv(path):
    """Lee y valida un CSV de mediciones RSSI."""
    df = pd.read_csv(path)
```

```

columnas_requeridas = {'x', 'y', 'piso', 'rssi'}
if not columnas_requeridas.issubset(df.columns):
    raise ValueError(f"El CSV '{path}' debe tener las columnas: {columnas_requeridas}."
    """
        f"Encontradas: {set(df.columns)}")
print(f"✅ CSV cargado: {path} -> {len(df)} filas, {df['piso'].nunique()} pisos")
print(f"    Pisos encontrados: {sorted(df['piso'].unique())}")
for p in sorted(df['piso'].unique()):
    sub = df[df['piso'] == p]
    print(f"    Piso {p}: {len(sub)} puntos | RSSI min={sub['rssi'].min()} dBm,
max={sub['rssi'].max()} dBm")
    return df
def generar_mapa_cuadrantes(df_piso, piso_num, ax, nombre_red):
    x = df_piso['x'].values
    y = df_piso['y'].values
    rssi = df_piso['rssi'].values
    RANGO = 35
    xi = np.linspace(-RANGO, RANGO, 200)
    yi = np.linspace(-RANGO, RANGO, 200)
    xi_grid, yi_grid = np.meshgrid(xi, yi)
    puntos = np.array(list(zip(x, y)))
    zi_grid = griddata(puntos, rssi, (xi_grid, yi_grid),
                      method='cubic', fill_value=rssi.min())
    if RANGO_AUTOMATICO:
        VMIN_ESCALA = float(np.percentile(rssi, 1))
        VMAX_ESCALA = float(np.percentile(rssi, 99))
    else:
        VMIN_ESCALA = VMIN_FIJO
        VMAX_ESCALA = VMAX_FIJO
    mascara_muerta = zi_grid < UMBRAL_ZONA_MUERTA
    labeled, num_features = label(mascara_muerta)
    contour = ax.contourf(xi_grid, yi_grid, zi_grid, levels=50,
                        cmap='RdYlGn_r', vmin=VMIN_ESCALA, vmax=VMAX_ESCALA, alpha=0.92)
    ax.contour(xi_grid, yi_grid, zi_grid, levels=[UMBRAL_ZONA_MUERTA],
              colors=['red'], linewidths=1.5, linestyles='--', alpha=0.8)
    ax.axhline(0, color='white', linewidth=0.8, alpha=0.5, linestyle=':')
    ax.axvline(0, color='white', linewidth=0.8, alpha=0.5, linestyle=':')
    # Puntos de medición coloreados por RSSI
    ax.scatter(x, y, c=rssi, cmap='RdYlGn_r', vmin=VMIN_ESCALA, vmax=VMAX_ESCALA,
              s=60, edgecolors='white', linewidth=1.2, zorder=5)
    for xi_pt, yi_pt, r in zip(x, y, rssi):
        offset_x = 0.4 if xi_pt >= 0 else -0.4
        ha = 'left' if xi_pt >= 0 else 'right'
        ax.text(xi_pt + offset_x, yi_pt, f'{r:.0f}', fontsize=6.5,
              color='white', ha=ha, va='center',
              bbox=dict(boxstyle='round,pad=0.15', fc='black', alpha=0.35, lw=0))
    ax.scatter(0, 0, c='gold', s=280, marker='*', edgecolors='black',
              linewidth=1.0, zorder=10, label='Router')
    ax.text(0.3, 0.6, 'Router\n(0,0)', fontsize=8, color='gold',
          fontweight='bold', va='bottom')
    ax.text(RANGO - 0.5, 0.3, '+X', fontsize=9, color='white', fontweight='bold')
    ax.text(-RANGO + 0.1, 0.3, '-X', fontsize=9, color='white', fontweight='bold')
    ax.text(0.3, RANGO - 0.5, '+Y', fontsize=9, color='white', fontweight='bold')
    ax.text(0.3, -RANGO + 0.3, '-Y', fontsize=9, color='white', fontweight='bold')
    for d in np.unique(np.abs(x[x != 0])):
        if abs(d) <= RANGO:
            ax.text(d, -0.9, f'{d:.0f}m', fontsize=6, color='lightgray', ha='center')
            ax.text(-d, -0.9, f'{d:.0f}m', fontsize=6, color='lightgray', ha='center')
    for d in np.unique(np.abs(y[y != 0])):
        if abs(d) <= RANGO:
            ax.text(0.3, d, f'{d:.0f}m', fontsize=6, color='lightgray', va='center')
            ax.text(0.3, -d, f'{d:.0f}m', fontsize=6, color='lightgray', va='center')
    if num_features > 0:
        cy_px, cx_px = center_of_mass(mascara_muerta)
        real_cx = xi[0] + cx_px * (xi[-1] - xi[0]) / mascara_muerta.shape[1]
        real_cy = yi[0] + cy_px * (yi[-1] - yi[0]) / mascara_muerta.shape[0]

```

```

        area_px = np.sum(mascara_muerta)
        radio = np.sqrt(area_px / np.pi) * (xi[-1] - xi[0]) / mascara_muerta.shape[1]
        radio = min(radio, RANGO * 0.4)
        circ = patches.Circle((real_cx, real_cy), radius=radio,
                               color='red', alpha=0.18, zorder=3)

        ax.add_patch(circ)
        ax.text(real_cx, real_cy, f'Zona muerta\n< {UMBRAL_ZONA_MUERTA} dBm',
                ha='center', va='center', color='red', fontsize=7.5, fontweight='bold',
                bbox=dict(boxstyle='round,pad=0.2', fc='black', alpha=0.4, lw=0))
    rssi_medio = rssi.mean()
    pct_muerta = np.sum(mascara_muerta) / mascara_muerta.size * 100
    ax.set_title(
        f'{nombre_red} - Piso {piso_num}\n'
        f'RSSI medio: {rssi_medio:.1f} dBm | Zona muerta: {pct_muerta:.1f}%',
        fontsize=9.5, color='white', pad=8)
    ax.set_xlabel('X (metros)', fontsize=9, color='lightgray')
    ax.set_ylabel('Y (metros)', fontsize=9, color='lightgray')
    ax.tick_params(colors='lightgray', labelsize=8)
    ax.set_xlim(-RANGO, RANGO)
    ax.set_ylim(-RANGO, RANGO)
    ax.set_aspect('equal')
    ax.grid(alpha=0.15, linestyle='--', color='white')
    for spine in ax.spines.values():
        spine.set_edgecolor('#444')
    return contour

def procesar_red(nombre_red, csv_path):
    """Genera la figura de cobertura (interpolación cúbica) para una red Wi-Fi completa."""
    print(f"\n{'='*60}\nProcesando red: {nombre_red}\n{'='*60}")
    df = leer_csv(csv_path)
    pisos = sorted(df['piso'].unique())
    n_pisos = len(pisos)
    ancho = 7 * n_pisos
    fig, axes = plt.subplots(1, n_pisos, figsize=(ancho, 7))
    fig.patch.set_facecolor('#1a1a2e')
    if n_pisos == 1:
        axes = [axes]
    contour_last = None
    for ax, piso in zip(axes, pisos):
        ax.set_facecolor('#0f0f1a')
        df_piso = df[df['piso'] == piso]
        contour_last = generar_mapa_cuadrantes(df_piso, piso, ax, nombre_red)
    cbar_ax = fig.add_axes([0.92, 0.12, 0.985, 0.76])
    cbar = fig.colorbar(contour_last, cax=cbar_ax)
    cbar.set_label('RSSI (dBm)', color='lightgray', fontsize=10)
    cbar.ax.yaxis.set_tick_params(color='lightgray')
    cbar.set_ticks([-95, -90, -85, -80, -75, -70, -65, -60, -55])
    cbar.ax.set_yticklabels(
        ['-95', '-90', '-85', '-80', '-75\n(límite)', '-70', '-65', '-60', '-55'],
        color='lightgray', fontsize=7.5)
    fig.suptitle(f'Mapa de Cobertura Wi-Fi - 4 Cuadrantes por Piso - Red: {nombre_red}',
                fontsize=14, color='white', y=1.01, fontweight='bold')
    ruta_salida = f"mapa_rssi_{nombre_red.lower().replace(' ', '_')}.png"
    plt.tight_layout(rect=[0, 0, 0.91, 1])
    plt.savefig(ruta_salida, dpi=200, bbox_inches='tight',
                facecolor=fig.get_facecolor())
    print(f"\n📁 Imagen guardada en: {ruta_salida}")
    plt.close(fig)
    return ruta_salida

if __name__ == '__main__':
    resultados = {}
    for nombre_red, csv_path in ARCHIVOS.items():
        resultados[nombre_red] = procesar_red(nombre_red, csv_path)
    print(f"\n{'='*60}\nResumen de archivos generados:\n{'='*60}")
    for red, ruta in resultados.items():
        print(f" {red}: {ruta}")

```